These statements create a temp.users table on the fly from a SELECT statement inside the temp schema you already have or will create. The temp.users table is similar to the Rideshare users table but with fewer columns. You'll add indexes covering some of these columns.

To start, there will be no indexes on the temp.users table. This makes it faster to insert rows because no Index maintenance will be performed as rows are inserted, updated, or deleted.

Once you have the table created and populated with 10 million rows, you're ready to begin creating experiments.

When filtering rows for tables with low row counts, PostgreSQL may decide to scan the whole table instead of using an Index you've created when it needs to find a few rows for a query. This can be surprising behavior but makes sense as you learn more about Indexing.

You may already know that Index Scans will outperform a Sequential Scan (table scan) when filtering for a row or two. You may not know that when items are found in an index and the PostgreSQL query plan shows an "Index Scan", that PostgreSQL doesn't usually get all of the information needed solely from the Index. PostgreSQL still needs to access the "Heap"[2] (which is the table row data) after the Index most of the time. That means an Index Scan might be two scans, a Index Scan and a Heap Scan. The combination of those two scans might be slower than scanning the table directly. This is dependent on many factors.

The most optimal scan type is an Index Only Scan. This happens when all of the fields needed either from a SELECT clause or a filtering option are contained in the Index itself. In that case a Heap scan is not necessary, which saves a considerable amount of time.

Enabling Index Only Scans is one of the reasons you saw earlier to limit the fields you're requesting in your Active Record query code.

With those explanations in place, you're now ready to begin experimenting with the temp.users table you've just created.

From psql run this query:

```
SELECT * FROM temp.users
WHERE last_name IN ('lname10000', 'lname100000', 'lname1000000');
```

2. https://www.postgresql.org/docs/current/glossary.html